



Deresegn POS

Technical Specifications (TDD/LLD)

Technical Design Document

Document Version: 1.0

Application Version: 1.0.3+4

Date: August 5, 2026

Classification: Confidential

Prepared by: Micro Sun & Solution PLC

Email: amanuielt@mssethiopia.com

Phone: +251 911 058 179



Deresegn POS - Technical Specifications (TDD / LLD)

1. Application Structure

1.1 Architecture Pattern

The mobile application follows a Model-Controller-Service pattern using the GetX framework for reactive state management:

- Models: Pure Dart classes representing data structures. Handle JSON serialization/deserialization with `toJson()` and `fromJson()` factory constructors.
- Controllers: `GetxController` subclasses managing reactive state (`obs` variables) and business logic. Bound to the widget tree via `Get.put()` or `Get.lazyPut()`.
- Services: Stateless utility classes for API communication, PDF generation, and local storage.
- Screens: Flutter `StatelessWidget` or `StatefulWidget` classes composing the UI. They read state from controllers and delegate actions.

1.2 Project Structure

```
lib/
+-- main.dart                # App entry point, initializes services
+-- config/
|   +-- app_settings.dart    # User-configurable settings (cashier name, etc.)
|   +-- config_preference.dart # Secure credential storage wrapper
|   +-- dio_config.dart      # HTTP client configuration, interceptors
+-- controllers/
|   +-- auth_controller.dart  # Company/Branch/MoR authentication
|   +-- invoice_controller.dart # Invoice creation, submission, PDF logic
|   +-- invoice_history_controller.dart # Invoice listing with pagination
|   +-- receipt_controller.dart # Receipt registration
|   +-- sync_controller.dart  # Connectivity sync coordination
+-- models/
|   +-- auth_models.dart      # LoginRequest, BranchLoginRequest, etc.
|   +-- company_models.dart   # CompanyRegisterRequest, CompanyLoginRequest
|   +-- invoice_models.dart    # InvoiceRegisterRequest, SupplyItem, QueuedInvoice
|   +-- invoice_history_model.dart # InvoiceSummary, paginated response
|   +-- receipt_models.dart    # ReceiptRegisterRequest, WithholdingReceiptRequest
+-- screens/
|   +-- branch_setup_screen.dart # Branch creation + cert upload UI
|   +-- company_auth_screen.dart # Company registration/login UI
|   +-- dashboard_screen.dart   # Bottom tab navigation host
|   +-- invoice_detail_screen.dart # Single invoice detail view
|   +-- invoice_generator_screen.dart # Invoice creation form
|   +-- invoice_history_screen.dart # Paginated invoice list
|   +-- pdf_preview_screen.dart # PDF preview and sharing
|   +-- settings_screen.dart    # App settings UI
```



```
| +-- setup_terminal_screen.dart      # MoR terminal setup / splash
| +-- supplies_screen.dart          # Product catalog CRUD
+-- services/
| +-- api_service.dart              # Static API call wrappers
| +-- dio_service.dart              # Low-level HTTP methods (GET, POST, DELETE)
| +-- invoice_pdf_service.dart      # Invoice PDF generation
| +-- offline_queue_service.dart    # Offline queuing + auto-sync
| +-- receipt_pdf_service.dart      # Receipt PDF generation
+-- theme/
| +-- app_theme.dart                # Light/Dark ThemeData definitions
| +-- theme_service.dart            # Theme mode toggle logic
+-- utils/
| +-- app_translations.dart         # English/Amharic translations
+-- initial_navigation_middleware.dart # Route guard for initial navigation
```

2. Local Storage Usage

2.1 flutter_secure_storage (Encrypted)

Used for all sensitive credentials. On Android, data is encrypted using AES-256 backed by the Android Keystore. On iOS, data is stored in the Keychain.

Key	Data Stored	Sensitivity
`access_token`	Branch JWT token	High
`refresh_token`	MoR refresh token	High
`company_access_token`	Company owner JWT token	High
`mor_token`	MoR API access token	High
`client_id`	MoR client ID	High
`client_secret`	MoR client secret	High
`api_key`	MoR API key	High
`tin`	Branch TIN number	Medium
`company_id`	Company identifier	Low
`branch_id`	Branch identifier	Low
`is_dark_mode`	Theme preference	Low

2.2 shared_preferences (Unencrypted)

Used for non-sensitive application state and the offline invoice queue.

Key	Data Stored	Sensitivity
`offline_invoice_queue`	JSON-serialized array of pending invoices	Medium - Contains buyer details, amounts, and invoice payloads

2.3 cookie_jar / PersistCookieJar

HTTP cookies are persisted to the application documents directory under `.cookies/`. Used for maintaining HTTP session state with the backend API.

Location	Data Stored	Sensitivity
`documents_dir`/.cookies/	HTTP session cookies	Medium

3. Sensitive Data Handling

3.1 Credentials on Device

All authentication tokens and MoR API credentials are stored exclusively via `flutter\_secure\_storage`. The `ConfigPreference` class acts as the sole gateway:

- Tokens are loaded into static memory variables at app startup (`ConfigPreference.init()`).
- Tokens are never logged in their raw form.
- When clearing sessions, all token keys are explicitly deleted from secure storage.
- Default fallback values for MoR credentials exist in the source code for development purposes (see Section 3.3).

3.2 Credentials on Backend

- `client\_secret` and `api\_key` fields on the `CompanyBranch` model use Laravel's `encrypted` cast, which provides transparent AES-256-CBC encryption/decryption using the application's `APP\_KEY`.
- Private keys (`.key` files) are stored on the filesystem at `storage/app/mor-credentials/{company\_id}/{branch\_slug}/private\_key.key` with file permissions restricted to `0600`.
- Certificates (`.pem` files) are stored alongside private keys with the same restricted permissions.
- Passwords are stored as bcrypt hashes.

3.3 Hardcoded Default Values (Development)

The `ConfigPreference` class contains fallback default values for development/testing purposes:

Method	Default Value	Risk
`getClientId()`	`127ae9ad-8de2-4856-ba88-4e6a49ad10d0`	Low - Only used if secure storage is empty
`getClientSecret()`	`d3ddb848-9daa-44ab-8d96-374fcc8c9e6b`	Medium - Should be removed in production
`getApiKey()`	`dc481579-a6e7-4594-abcf-5493e261685e`	Medium - Should be removed in production
`getTin()`	`0000037187`	Low - Testing TIN

Note: These defaults are used as fallbacks when the secure storage does not contain values. In normal operation, credentials are provisioned during branch setup and override these defaults.

4. API Communication

4.1 Base Configuration

- Base URL: `https://api.deresegn.com`
- Protocol: HTTPS (TLS 1.2+)
- Content Type: `application/json` (default), `multipart/form-data` (branch creation)



- Connect Timeout: 60 seconds
- Receive Timeout: 120 seconds

4.2 API Endpoints

Method	Endpoint	Auth Required	Description
POST	`/api/company/register`	None	Register a new company
POST	`/api/company/login`	None	Company owner login
GET	`/api/companies`	None	List active companies
POST	`/api/branch/login`	None	Branch operator login
POST	`/api/branches`	Company JWT	Create a new branch
POST	`/api/login`	Branch JWT + Branch-Id	MoR login
POST	`/api/refresh/token`	None (refresh token in body)	Refresh MoR token
POST	`/api/invoice/register`	Branch JWT + MoR Token	Register invoice with MoR
POST	`/api/cancel/invoice`	Branch JWT + MoR Token	Cancel invoice
POST	`/api/irn/verify`	Branch JWT + MoR Token	Verify invoice by IRN
POST	`/api/receipt/sales`	Branch JWT + MoR Token	Register sales receipt
POST	`/api/receipt/withholding`	Branch JWT + MoR Token	Register withholding receipt
GET	`/api/invoices`	Branch JWT	List invoices (paginated)
GET	`/api/invoices/{id}`	Branch JWT	Get single invoice
GET	`/api/receipts/{type}/{value}`	Branch JWT	Get receipt by RRN/MOR/IRN
GET	`/api/supplies`	Branch JWT	List product catalog
POST	`/api/supplies`	Branch JWT	Create product
POST	`/api/supplies/{id}`	Branch JWT	Update product
DELETE	`/api/supplies/{id}`	Branch JWT	Delete product

4.3 Request Headers

Header	Value	When Sent
`Content-Type`	`application/json`	All requests
`Authorization`	`Bearer {branch_jwt}`	All authenticated requests
`Branch-Id`	`{branch_id}`	All requests (injected by AuthInterceptor)
`Mor-Token`	`Bearer {mor_access_token}`	MoR-proxied requests (not `/api/login`)

4.4 Retry Logic

The `DioService` implements retry logic for GET requests:

- Max retries: 2 attempts
- Retry conditions: HTTP 520 or response contains `retryable: true`
- Backoff strategy: `retry\_after` from response body, or `attempt \* 2` seconds
- Maximum delay: 5 seconds (capped)

5. Certificate Handling



5.1 TLS Configuration

- The mobile app relies on the platform's default TLS implementation for HTTPS connections.
- No custom certificate pinning is currently implemented in the Dio configuration.
- The backend validates standard TLS certificates for connections to the MoR API.

5.2 MoR Digital Certificates

Each branch maintains its own PKI credentials:

- Private Key: RSA private key file (``private_key.key``), stored server-side at ``storage/app/mor-credentials/{company_id}/{branch_slug}/``.
- Certificate: X.509 certificate file (``certificate.pem``), stored alongside the private key.
- File Permissions: Set to ``0600`` (owner read/write only) upon upload.
- Upload: Performed during branch creation via ``multipart/form-data`` upload from the mobile app.
- Usage: The ``MorApiServiceTrait`` loads the private key using ``openssl_pkey_get_private()``, signs payloads with ``openssl_sign()`` (SHA-512), and attaches the base64-encoded certificate.

6. Authentication Mechanisms

6.1 Company Owner Authentication

- Method: Phone number + password + company ID
- Token: JWT issued by ``tymon/jwt-auth`` from the ``CompanyUser`` model
- Guard: ``auth:api`` (default Laravel JWT guard)
- Validation: Laravel ``Validator`` with required phone, password (min:6), nullable company_id

6.2 Branch Authentication

- Method: TIN number + password
- Token: JWT issued by ``tymon/jwt-auth`` from the ``CompanyBranch`` model (implements ``JWTSubject``)
- Guard: ``branch`` (custom guard)
- Validation: The backend iterates over all branches with the given TIN and checks bcrypt password hash against each candidate (TIN is not unique across branches of the same company)
- Status Check: Branch must have ``status === 'active'``

6.3 MoR Authentication

- Method: Client ID, client secret, API key, and TIN
- Token: MoR-issued access token + refresh token (with expiration)
- Refresh: Automatic on 401 (code ``4503`` or message ``GATEWAY ERROR``) via ``AuthInterceptor.onError()``
- Refresh Lock: A static ``Future<String?>? _refreshFuture`` prevents duplicate concurrent refresh requests



7. Session Management

7.1 Token Lifecycle

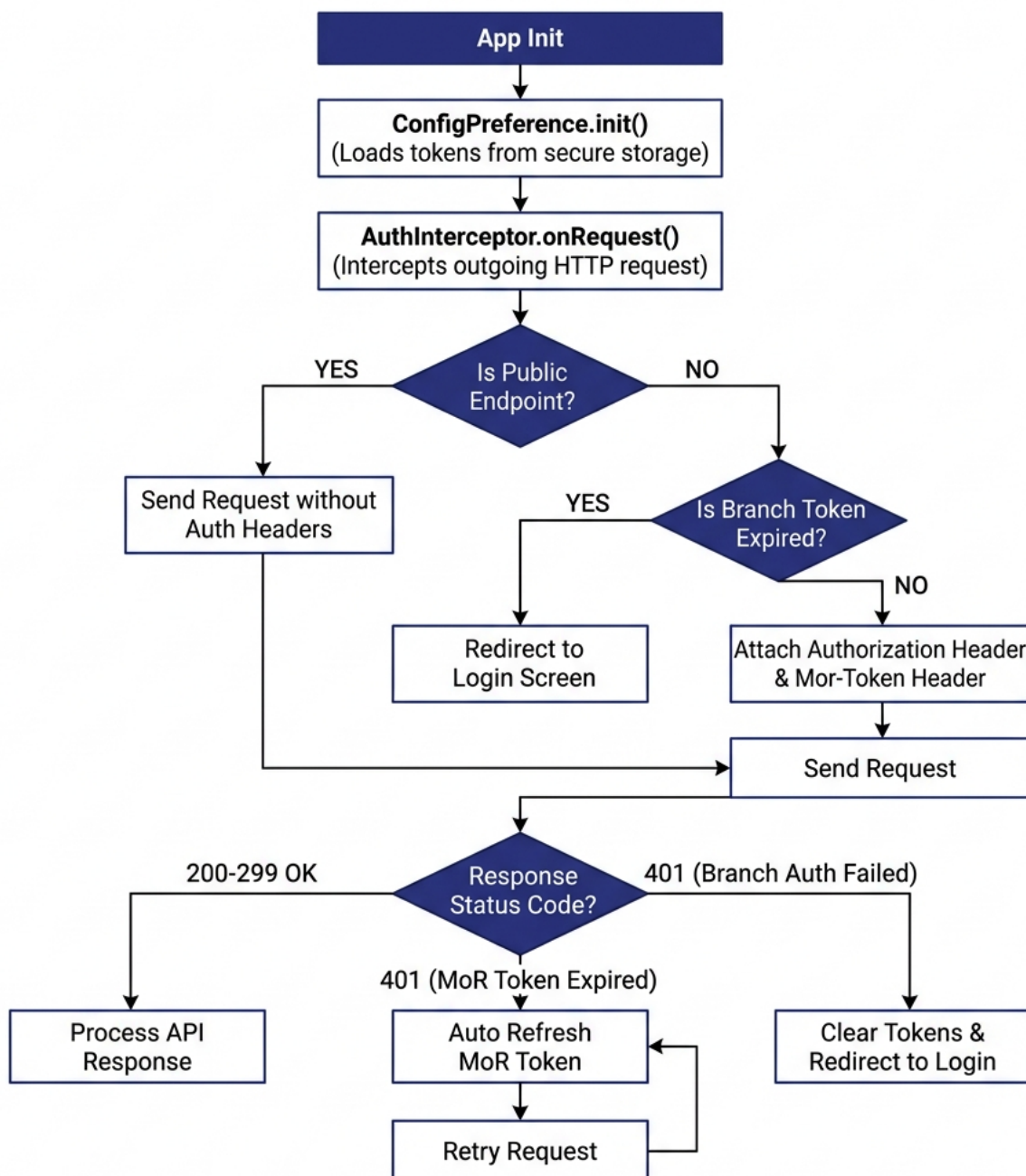


Figure: Authentication & Token Lifecycle Flow



7.2 Session Expiry Handling

- Branch JWT Expiry: Detected by ``JwtDecoder.isExpired()`` before each request. If expired, user is redirected to login.
- MoR Token Expiry: Detected reactively when the MoR returns HTTP 401 with code ``4503`` or ``GATEWAY ERROR``. The interceptor attempts automatic refresh.
- Refresh Failure: If refresh fails, user is redirected to login.
- Logout: ``ConfigPreference.clearTokens()`` removes all stored tokens and redirects to the company auth screen.

8. Data Models

8.1 Mobile Models

Model	Fields	Purpose
<code>`LoginRequest`</code>	clientId, clientSecret, apiKey, tin	MoR login credentials
<code>`BranchLoginRequest`</code>	tinNumber, password	Branch authentication
<code>`CompanyRegisterRequest`</code>	companyName, tinNumber, ownerName, phone, password, email, website	Company registration
<code>`CompanyLoginRequest`</code>	phone, password, companyId	Company owner login
<code>`InvoiceRegisterRequest`</code>	documentDetails, transactionType, sourceSystem, sellerDetails, buyerDetails, itemList, valueDetails, paymentDetails, referenceDetails, version	Invoice registration payload
<code>`InvoiceCancelRequest`</code>	irn, reasonCode	Invoice cancellation
<code>`InvoiceVerificationRequest`</code>	irn	Invoice verification
<code>`SupplyItem`</code>	id, itemCode, productDescription, natureOfSupplies, unitPrice, unit, taxCode, discount, exciseTaxRate, isExciseTaxable	Product catalog item
<code>`QueuedInvoice`</code>	id, request, buyerName, grandTotal, queuedAt	Offline-queued invoice
<code>`ReceiptRegisterRequest`</code>	receiptNumber, receiptType, reason, receiptDate, receiptCounter, sourceSystemType, sourceSystemNumber, receiptCurrency, collectedAmount, sellerTIN, invoices, transactionDetails	Sales receipt
<code>`WithholdingReceiptRequest`</code>	receiptNumber, reason, receiptCounter, sourceSystemType, sourceSystemNumber, buyerTIN, invoiceDetail, withholdDetail	Withholding receipt
<code>`InvoiceSummary`</code>	id, irn, documentNumber, status, buyer, seller, totals, items, createdAt, requestPayload	Invoice display

8.2 Backend Models

Model	Table	Key Fields	Casts
<code>`Company`</code>	companies	id, company_legal_name, tin_number, email, expired_date	-
<code>`CompanyUser`</code>	company_users	id, company_id, name, phone, password, role	-
<code>`CompanyBranch`</code>	company_branches	id, company_id, branch_name, tin_number, client_id, client_secret, api_key, password, private_key_path, certificate_path, status	client_secret->encrypted, api_key->encrypted
<code>`MorInvoice`</code>	mor_invoices	id, branch_id, company_id, irn, document_number, status, ack_date, signed_qr, request_payload, buyer/seller details, values, item_list	request_payload->array, item_list->array, ack_date->datetime, decimal fields



Model	Table	Key Fields	Casts
`MorInvoiceAudit`	mor_invoice_audits	branch_id, company_id, tin, document_number, request_payload, response_payload, response_status_code, success, error_message	-
`MorLoginAudit`	mor_login_audits	branch_id, company_id, client_id, tin, request_payload, response_payload, status, success	-
`MorReceipt`	mor_receipts	id, mor_id, status, rrn, invoice_irn, receipt_number, qr, is_success, request_payload	-
`Supply`	supplies	id, item_code, product_description, nature_of_supplies, unit_price, unit, tax_code, discount, excise_tax_rate, is_excise_taxable	-

9. Input Validation

9.1 Mobile-Side Validation

Field	Rule
Buyer Name	Required, non-empty string
Invoice Items	At least one item required
TIN (Branch Login)	Required, non-empty string
Password (Branch Login)	Required, non-empty string
Company Phone	Required, non-empty string
Company Password	Required, non-empty string
Company ID	Required, non-empty string

9.2 Backend Validation (Laravel Validator)

Endpoint	Field	Rule
Company Register	phone	required, string
Company Register	password	required, string, min:6
Company Login	phone, password	required, string
Company Login	company_id	nullable, integer
Branch Create	branch_name	required, string, max:255
Branch Create	tin_number	required, string, max:50
Branch Create	client_id, client_secret, api_key	required, string, max:255
Branch Create	password	required, string, min:6
Branch Create	private_key	required, file, max:10240
Branch Create	certificate	required, file, max:10240
Branch Login	tin_number	required, string
Branch Login	password	required, string

10. Business Logic



10.1 Tax Calculation Engine (Mobile)

The `InvoiceItem` class implements the following calculation chain per line item:

```
Net Value = Unit Price × Quantity
Discount Amount = Fixed discount value
After Discount = Net Value - Discount Amount
Excise Amount = (After Discount × Excise Rate) if excise taxable, else 0
VAT Base = After Discount + Excise Amount
VAT Amount = VAT Base × Tax Rate
Total Line Amount = VAT Base + VAT Amount
```

Supported tax categories:

- `VAT15` - 15% VAT
- `TOT2` - 2% Turnover Tax
- `TOT10` - 10% Turnover Tax
- `EXMT` - Tax Exempt (0%)
- `ZERO` - Zero Rated (0%)

10.2 Document Number Sequencing

The app auto-generates document numbers by:

- Fetching the latest invoice history.
- Finding the maximum existing document number.
- Incrementing by 1.
- If history is unavailable, fallback to `DateTime.now().millisecondsSinceEpoch % 100000`.
- If MoR rejects with HTTP 406 and provides an expected value, auto-retry with the corrected number.

10.3 Document Types

- `INV` - Standard Invoice (Cash Sale, Credit Sale)
- `CRE` - Credit Note (requires reference IRN)
- `DEB` - Debit Note (requires reference IRN)

10.4 Transaction Types

- `B2B` - Business-to-Business
- `B2C` - Business-to-Consumer

11. Error Handling

11.1 Network Errors

The `DioConfig` class translates `DioException` types to user-friendly messages:

DioExceptionType	Message
------------------	---------



DioExceptionType	Message
`cancel`	"Request cancelled"
`connectionTimeout`	"Connection timeout"
`sendTimeout`	"Send timeout"
`receiveTimeout`	"Receive timeout"
`badResponse`	"HTTP error {code}: {message}"
`badCertificate`	"Bad certificate, try switching devices"
`connectionError`	"Connection error, check your internet"
`unknown`	Exception message or "Other Dio error occurred"

11.2 API Error Handling

- Backend error responses contain `message` or `messages` fields.
- The `AuthController.\_parseMessage()` extracts and joins error messages for display.
- Errors are shown via `Get.snackbar()` at the bottom of the screen.

11.3 Session Expiry

- Detected proactively (JWT decode) and reactively (401 responses).
- Users are redirected to the appropriate login screen.

12. Root/Jailbreak Detection

No root/jailbreak detection is currently implemented in the application.

13. Code Obfuscation

Flutter's default release build configuration applies tree-shaking and Dart AOT compilation. No additional code obfuscation (e.g., `--obfuscate` flag) is documented as enabled in the current build configuration.

14. Logging Practices

14.1 Mobile Logging

The app uses the `logger` package for structured logging:

- `Logger().i()` - Informational messages (successful operations)
- `Logger().w()` - Warnings (unexpected but non-fatal conditions)
- `Logger().e()` - Errors (failed operations)
- `Logger().d()` - Debug messages (auth interceptor decisions)

The `LoggingInterceptor` logs all HTTP requests and responses:

- Requests: URL, method, headers, body (with pretty-printed JSON)
- Responses: Status code, URL, response body (pretty-printed)
- Errors: Status code, URL, error body (pretty-printed)

Note: The `LoggingInterceptor` logs full request/response bodies including headers with `Authorization` tokens. This interceptor is



disabled in test mode ('DioConfig.isTestMode').

14.2 Backend Logging

- Laravel's `Log` facade is used for error logging.
- MoR service exceptions are logged with branch ID and payload context.
- Audit tables capture structured request/response data.
- `clientSecret` is explicitly removed from audit payloads.
- `accessToken` and `refreshToken` are replaced with `MASKED` in login audit responses.